

Documento de Arquitetura

Sistema RAG-ANTT — Deploy como API REST
com LLM Local em Ambiente Kubernetes (Rancher)

Agência Nacional de Transportes Terrestres (ANTT)

DeepFeed

Fevereiro de 2026

Classificação: Documento de Arquitetura — Projeto Técnico

Descreve a arquitetura de referência para deploy do Sistema RAG-ANTT como API REST, com interface Streamlit para testes, LLM local via Ollama e integração com o sistema SIGESCANTT em ambiente Kubernetes gerenciado pelo Rancher.

Sumário

1	Introdução e Contexto	4
1.1	Princípios de Arquitetura	4
2	Visão Geral da Arquitetura	5
2.1	Diagrama de Componentes	5
3	Camada de Entrada — Ingress Controller	6
4	Componentes Detalhados	6
4.1	Pod RAG-API (FastAPI)	6
4.2	Pod Streamlit (Testes e Demonstrações)	8
4.3	Pod Ollama (LLM Local)	9
4.4	Pod SIGESCANTT (Sistema Externo)	15
4.5	FAISS Vectorstore — PersistentVolume	16
5	Contrato da API REST	16
5.1	Endpoints	17
5.2	Contrato do Endpoint Principal (POST /api/query)	17
6	Fluxo de Comunicação	19
6.1	Fluxo Principal — Consulta via SIGESCANTT	19
6.2	Fluxo Alternativo — Testes via Streamlit	20
7	Infraestrutura Kubernetes (Rancher)	21
7.1	Namespace e Services	21
7.2	Ingress Controller	22
7.3	Volumes Persistentes	22
8	Requisitos de Infraestrutura	23
8.1	Requisitos do Cluster Kubernetes	23
8.2	Viabilidade do Piloto sem GPU e Riscos Operacionais	24
8.3	Requisitos de Disco	25
9	Segurança e Isolamento	26
9.1	Camada Externa (Usuários)	26
9.2	Camada Interna (Cluster)	26
9.3	Compliance e Dados	27
10	Estratégia de Deploy	27
10.1	Ordem de Implantação	27

10.2 Fallback e Resiliência	28
11 Estimativa de Esforço	28
11.1 Delimitacao de Escopo para Cronograma	28
11.2 Estrutura de Etapas e Subetapas	29
11.3 Etapas Dependentes da ANTT	32
11.4 Escopo Não Incluído — Integração com Crawler	33
11.5 Resumo do Escopo	34
12 Conclusão	34

1 Introdução e Contexto

O Sistema RAG-ANTT (Retrieval-Augmented Generation) foi desenvolvido para permitir consultas inteligentes a documentos regulatórios da Agência Nacional de Transportes Terrestres. Atualmente, o sistema opera como uma aplicação monolítica Streamlit que integra interface de usuário, lógica de busca semântica (FAISS + sentence-transformers) e geração de respostas via provedores de LLM em nuvem (DeepSeek via OpenRouter, OpenAI).

O presente documento descreve a **arquitetura de referência** para a evolução deste sistema rumo a um ambiente de produção, contemplando:

1. **Separação de responsabilidades:** O RAG passa a expor uma API REST (FastAPI), desacoplada de qualquer interface gráfica;
2. **Manutenção da interface Streamlit:** O Streamlit permanece como ferramenta de testes e demonstrações, consumindo a mesma lógica central;
3. **LLM local:** Um modelo de linguagem é executado localmente via Ollama, eliminando dependência de APIs externas e custos por chamada;
4. **Integração com SIGESCANTT:** O sistema externo SIGESCANTT consome a API REST do RAG para oferecer funcionalidades de consulta inteligente aos seus usuários;
5. **Orquestração via Kubernetes (Rancher):** Todos os componentes são implantados como pods em um cluster Kubernetes gerenciado pelo Rancher.

1.1 Princípios de Arquitetura

A arquitetura proposta segue princípios fundamentais para ambientes regulatórios:

- **Desacoplamento:** Cada componente possui responsabilidade única e comunica-se via interfaces bem definidas (HTTP/REST);
- **Auditabilidade:** Toda consulta é rastreável — a API retorna fontes, trechos e metadados que fundamentam cada resposta;
- **Resiliência:** Falhas em componentes individuais (LLM, vectorstore) não comprometem o sistema inteiro;
- **Independência tecnológica:** O SIGESCANTT não precisa conhecer detalhes de IA, FAISS ou LLM — consome apenas JSON via HTTP;
- **Custo previsível:** Com LLM e embeddings locais, não há cobrança por chamada a APIs externas.

2 Visão Geral da Arquitetura

A arquitetura é composta por quatro pods principais, implantados em um único namespace Kubernetes (`rag-antt`), mais um volume persistente compartilhado para o índice vetorial:

Tabela 1: Componentes da arquitetura e suas responsabilidades

Pod	Tecnologia	Porta	Responsabilidade
<code>sgescantt</code>	Stack existente	XX	Interface principal do usuário final. Módulo de consulta RAG integrado ao sistema corporativo.
<code>rag-api</code>	FastAPI + Python	8000	Núcleo do RAG: busca semântica, orquestração de contexto, chamada ao LLM e formatação de respostas.
<code>streamlit</code>	Streamlit	8501	Interface visual para testes, demonstrações e depuração. Reutiliza a lógica central do RAG.
<code>ollama</code>	Ollama	11434	Servidor de inferência do LLM local. Expõe API compatível com OpenAI.

2.1 Diagrama de Componentes

O diagrama a seguir ilustra a disposição dos componentes e suas interações dentro do cluster Kubernetes. Cada caixa representa um pod isolado, e as setas indicam a direção do fluxo de dados:

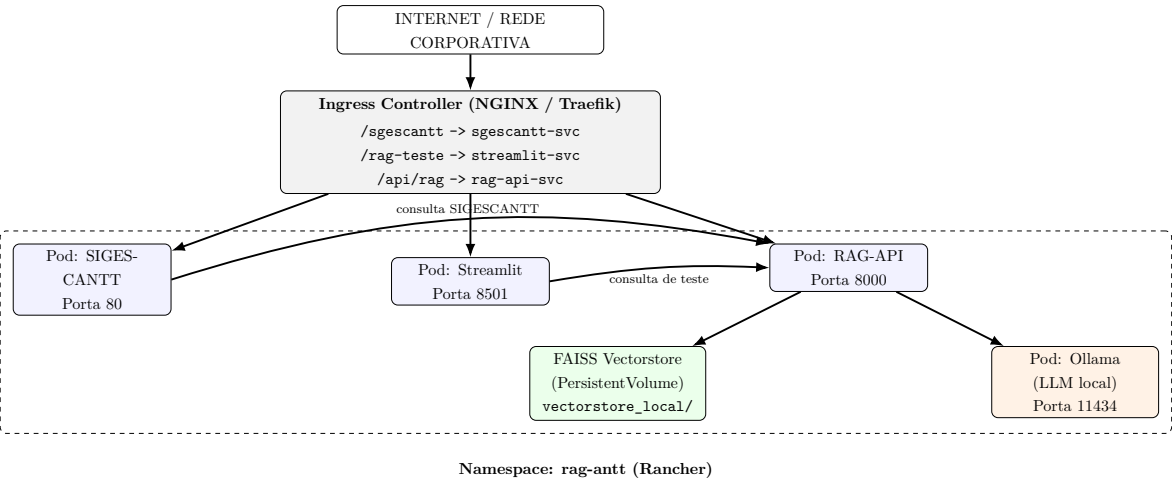


Figura 1: Diagrama de componentes no cluster Kubernetes (Rancher)

3 Camada de Entrada — Ingress Controller

O **Ingress Controller** é o único ponto de contato entre a rede externa (internet ou rede corporativa da ANTT) e os componentes internos do cluster. Ele funciona como um **proxy reverso inteligente** que:

- Recebe **todo** o tráfego HTTPS em uma única porta (443);
- Inspecciona o **path** da URL para decidir para qual serviço interno encaminhar;
- Centraliza regras de segurança, autenticação de borda e governança de acesso.

O Ingress também é responsável por **TLS** (certificados HTTPS), **CORS** (permitir que o SIGESCANTT chame a API de outro domínio) e **rate-limiting** (proteger contra uso abusivo). No Rancher, o Ingress Controller já vem pré-configurado — normalmente NGINX ou Traefik. As regras de roteamento por *path* estão consolidadas na Tabela 15.

Esse ponto único de entrada garante que **nenhum componente interno** (API, Ollama, vectorstore) fique exposto diretamente à rede externa. Todo tráfego é filtrado, autenticado e roteado pelo Ingress antes de chegar aos pods.

4 Componentes Detalhados

4.1 Pod RAG-API (FastAPI)

Este é o **componente central** da arquitetura. Ele encapsula toda a lógica de recuperação semântica e orquestração, expondo-a como uma API REST padrão. Internamente, o que ele faz a cada requisição é:

1. Recebe a pergunta via POST `/api/query`;
2. Gera o embedding da pergunta usando `LocalEmbeddings` (a classe que já existe em `llm_providers.py`, usando `sentence-transformers all-MiniLM-L6-v2`);
3. Busca no FAISS usando `pesquisar_documentos()` — a mesma função que hoje o Streamlit chama, com busca MMR + busca por keywords + reranking;
4. Monta o contexto com os trechos encontrados (tipo, número, ano, caminho, conteúdo);
5. Chama o LLM via `gerar_resposta()` — que usa `ChatOpenAI` com `base_url` apontando para o Ollama;
6. Retorna JSON com a resposta, fontes e métricas.

Responsabilidades

- Carregar e manter em memória o índice vetorial FAISS;
- Gerar embeddings das consultas utilizando sentence-transformers (local, gratuito);
- Executar busca por similaridade semântica no vectorstore;
- Montar o contexto (prompt + trechos relevantes) e enviar ao LLM;
- Formatar a resposta com citações, fontes e metadados;
- Expor endpoints REST para consulta, status, reindexação e listagem de documentos.

Relação com o Código Atual

O ponto chave é que **o código já existe**. O pod RAG-API reutiliza as funções já implementadas no projeto. O `LLMManager` em `llm_providers.py` já usa `ChatOpenAI` com `base_url` configurável — para apontar para o Ollama, basta adicionar um novo provedor em `config.py` com `base_url: "http://ollama-svc:11434/v1"`. A mesma classe `ChatOpenAI` funciona sem alteração.

Tabela 2: Mapeamento entre funções existentes e endpoints da API

Função existente	Endpoint	Descrição
<code>carregar_vectorstore_com_provider()</code>	<code>startup</code>	Carrega o índice FAISS na inicialização do pod
<code>pesquisar_documentos()</code>	<code>POST /api/query</code>	Busca semântica + geração de resposta
<code>gerar_resposta()</code>	<code>POST /api/query</code>	Orquestração LLM com contexto
<code>create_llm_manager()</code>	<code>startup</code>	Inicializa o gerenciador de LLM
—	<code>GET /api/status</code>	Status dos componentes
—	<code>GET /api/health</code>	Health check para Kubernetes
—	<code>POST /api/reindex</code>	Dispara reindexação
—	<code>GET /api/documents</code>	Lista documentos indexados

A lógica central (busca semântica, geração de resposta, templates de prompt) **não precisa ser reescrita**. O FastAPI atua como uma camada fina de exposição HTTP sobre as funções já existentes em `antt_rag_unified.py` e `llm_providers.py`.

Configuração do Container

Tabela 3: Especificações do container RAG-API

Parâmetro	Valor
Imagem base	<code>python:3.10-slim</code>
Porta exposta	8000
Memória mínima	2 GB
Memória recomendada	4 GB
CPU mínima	2 cores
Healthcheck	<code>GET /api/health</code>
Volume montado	<code>/app/vectorstore_local/</code> (PVC)

4.2 Pod Streamlit (Testes e Demonstrações)

O Streamlit permanece como interface visual, executando a mesma aplicação `antt_rag_unified.py` existente. Ele **não foi removido** da arquitetura — continua disponível para a equipe técnica testar consultas, visualizar métricas, inspecionar documentos e depurar problemas. Sua função é dupla:

- Testes internos:** A equipe de desenvolvimento utiliza o Streamlit para validar consultas, inspecionar métricas de busca e depurar o comportamento do RAG;
- Demonstrações:** Gestores e *stakeholders* podem visualizar o sistema em funcionamento sem necessidade de integração com o SIGESCANTT.

A diferença entre o Streamlit e a API é apenas a **porta de entrada**: o Streamlit chama as funções Python diretamente (como `pesquisar_documentos()` e `gerar_resposta()`), sem passar pela API HTTP. Já o SIGESCANTT passa pela API. No deploy, o Streamlit roda em **seu próprio pod**, isolado dos demais.

Coexistência com a API

O Streamlit e a API REST **coexistem sem conflito**. Ambos compartilham:

- O mesmo volume persistente (vectorstore FAISS);
- O mesmo pod Ollama para geração de respostas;

- A mesma lógica central (funções Python do projeto).

Tabela 4: Comparação entre as duas portas de entrada

Porta	Interface	Quem usa
8501	Streamlit (visual)	Equipe de desenvolvimento, QA, demonstrações
8000	FastAPI (JSON)	SIGESCANTT, sistemas externos, automações

Por que manter os dois? Porque servem públicos diferentes: o Streamlit é para **testes visuais, demonstrações para gestores e debug rápido**; a API REST é para **integração com o SIGESCANTT e qualquer outro sistema**. Se o Streamlit ficar indisponível, a produção (SIGESCANTT via API) não é afetada.

4.3 Pod Ollama (LLM Local)

O Ollama é um servidor de inferência que executa modelos de linguagem localmente, expondo uma API **100% compatível com o padrão OpenAI** (`/v1/chat/completions`). Isso significa que o código do RAG — que já utiliza `ChatOpenAI` do LangChain — pode apontar para o Ollama sem alterações estruturais, bastando alterar o parâmetro `base_url`.

Compatibilidade com o Código Existente

O `LLMManager` em `llm_providers.py` já utiliza o `ChatOpenAI` com `base_url` configurável. Atualmente, o `config.py` aponta para o `OpenRouter`:

Listing 1: Configuração atual em `config.py` — DeepSeek via OpenRouter

```
"deepseek": {
    "name": "DeepSeek (via OpenRouter)",
    "base_url": "https://openrouter.ai/api/v1",
    ...
}
```

Para apontar para o Ollama, basta adicionar um novo provedor:

Listing 2: Configuração para Ollama — mudança de URL apenas

```
"ollama": {
    "name": "Ollama (LLM Local)",
    "base_url": "http://ollama-svc:11434/v1",
    "models": {
        "qwen2.5:7b": "qwen2.5:7b",
        "llama3.1:8b": "llama3.1:8b",
    }
}
```

```
    },  
    "get_api_key": lambda: "ollama", # placeholder  
    ...  
}
```

A classe `ChatOpenAI` do LangChain aceita qualquer endpoint compatível com a API OpenAI. O Ollama, o LM Studio, o vLLM e o llama.cpp expõem exatamente essa API. A transição de provedor em nuvem para LLM local é, portanto, uma **mudança de configuração**, não de arquitetura.

Por que um Pod Separado?

O LLM consome muita memória e, opcionalmente, GPU. Separando-o em seu próprio pod:

- Ele pode ser **escalado independentemente** dos demais componentes;
- Pode ser colocado em um **node com GPU** no cluster, enquanto os outros pods rodam em nodes comuns;
- Se o LLM travar ou consumir memória em excesso, os outros pods continuam funcionando;
- O Kubernetes pode aplicar *resource limits* específicos para o pod do Ollama sem afetar a API ou o Streamlit.

O Ollama **nunca é exposto** para fora do cluster. O Service `ollama-svc:11434` é do tipo **ClusterIP** — somente os pods internos (RAG-API e Streamlit) conseguem acessá-lo. Isso impede que alguém de fora use o modelo indevidamente.

Modelos Recomendados

A seleção de modelo deve considerar a tarefa específica do RAG: compreender documentos regulatórios em português brasileiro, interpretar trechos técnicos e gerar respostas estruturadas com citações. A Tabela 5 apresenta os modelos recomendados:

Tabela 5: Modelos recomendados para o RAG-ANTT via Ollama

Modelo	Parâmetros	Disco	RAM	Observações
Qwen2.5:7b	7B	4,7 GB	8 GB	Excelente em português. Melhor relação qualidade/recurso.
Llama 3.1:8b	8B	4,7 GB	8 GB	Bom multilinguismo. Amplamente testado.
Mistral:7b	7B	4,1 GB	8 GB	Bom desempenho geral. Rápido em CPU.
Gemma2:9b	9B	5,4 GB	10 GB	Alta qualidade. Requer mais recursos.
Phi3:mini	3,8B	2,3 GB	4 GB	Muito leve. Qualidade inferior, mas funcional.

Recomendação primária: Qwen2.5:7b, por sua excelente compreensão de português e tamanho moderado. Para ambientes com recursos muito restritos, Phi3:mini é uma alternativa viável com qualidade reduzida.

Impacto da GPU na Latência — Análise Detalhada

O Ollama utiliza internamente o `llama.cpp` como backend de inferência, que opera modelos quantizados no formato GGUF (tipicamente Q4_K_M, com 4 bits por peso). A performance de geração de texto depende de duas fases distintas:

- **Prompt processing (pp):** Processamento do contexto de entrada (pergunta + trechos de documentos). Essa fase é altamente paralelizável e beneficia-se enormemente de GPU;
- **Text generation (tg):** Geração token a token da resposta. É sequencial por natureza, mas GPU ainda oferece ganho significativo devido à largura de banda de memória.

No contexto do RAG-ANTT, cada consulta envolve tipicamente:

- **Entrada (prompt):** 800 a 2.000 tokens (pergunta + 5–8 trechos de documentos + template de instrução);
- **Saída (resposta):** 150 a 400 tokens (resposta estruturada com citações).

Tabela 6: Latência esperada por modo de execução — modelo 7B quantizado (Q4_K_M)

Modo de execução	pp (t/s)	tg (t/s)	Latência total	Observações
CPU — 4 cores (servidor básico)	8–15	4–8	45–90 s	Uso mínimo viável
CPU — 8 cores (servidor dedicado)	15–30	8–15	20–50 s	Aceitável para uso interno
GPU NVIDIA T4 (16 GB VRAM)	150–250	25–40	6–12 s	GPU de entrada para servidores
GPU NVIDIA RTX 4090 (24 GB VRAM)	400–600	80–120	2–4 s	Alta performance
GPU NVIDIA A100 (80 GB VRAM)	600–1.000	100–150	1,5–3 s	Produção de alta escala

Nota metodológica: Os valores de *pp* (prompt processing) e *tg* (text generation) são expressos em tokens por segundo. A latência total é calculada para uma consulta típica de 1.500 tokens de entrada e 250 tokens de saída: $t_{\text{total}} = \frac{1500}{\text{pp}} + \frac{250}{\text{tg}}$. Os valores são baseados em benchmarks públicos do `llama.cpp` (backend do Ollama) para modelos 7B quantizados em Q4_K_M, reportados pela comunidade em hardware equivalente.

Importante — variabilidade dos valores: Os números apresentados representam medianas observadas em condições controladas (benchmark isolado, sem concorrência, com o modelo já carregado em memória). Na operação real, a latência pode ser **1,5× a 2× maior** que os valores tabelados, dependendo de fatores como:

- **Tamanho do prompt:** Consultas com documentos longos (ex.: tabelas extensas ou múltiplos normativos) podem gerar prompts de 3.000–4.000 tokens, dobrando o tempo de prompt processing;
- **Concorrência:** Se múltiplas consultas chegam simultaneamente, elas são enfileiradas — a segunda consulta espera a primeira terminar (em CPU) ou compartilha recursos (em GPU com batching);
- **Cold start:** Na primeira consulta após inicialização do pod, o modelo precisa ser carregado do disco para a memória (5–15 segundos adicionais). Consultas subsequentes não sofrem esse overhead;
- **Throttling térmico:** Em servidores sem refrigeração adequada, CPUs e GPUs

podem reduzir a frequência de clock sob carga sustentada, degradando a performance em 10–20%;

- **Comprimento da resposta:** Respostas mais elaboradas (400+ tokens) aumentam proporcionalmente o tempo de geração.

Portanto, os valores devem ser interpretados como **estimativas otimistas de caso típico**. Para dimensionamento de infraestrutura, recomenda-se aplicar um fator de segurança de **1,5×** sobre os valores tabelados (ex.: se a tabela indica 35 s em CPU, planejar para 50–55 s em pico).

A diferença entre execução em CPU e GPU é de **uma a duas ordens de grandeza**, dependendo da fase. Em termos práticos:

Tabela 7: Comparação direta CPU vs. GPU para uma consulta RAG típica

Métrica	CPU (8 cores)	GPU (RTX 4090)	Ganho
Prompt processing	~100 ms/tok	~2 ms/tok	~50×
Text generation	~100 ms/tok	~10 ms/tok	~10×
Latência total (consulta RAG)	~35 s	~3 s	~12×
Consultas simultâneas	1 (sequencial)	2–4 (batching)	Escalabilidade

Latência Ponta a Ponta do Pipeline RAG

A inferência do LLM é o **gargalo dominante** do pipeline. Os demais componentes contribuem com latência marginal:

Tabela 8: Decomposição da latência por etapa do pipeline RAG

Etapa	Latência típica	% do total	Observação
Embedding da pergunta (sentence-transformers)	50–150 ms	<1%	Local, CPU do pod RAG-API
Busca FAISS (similaridade vetorial)	10–50 ms	<1%	Índice em memória RAM
Reranking + montagem de contexto	20–80 ms	<1%	Processamento Python puro
Inferência LLM (Ollama) — CPU	20–50 s	>99%	Gargalo absoluto
Inferência LLM (Ollama) — GPU (T4 ou superior)	2–12 s	>95%	Ainda dominante; varia conforme classe de GPU
Formatação da resposta + JSON	5–15 ms	<1%	Overhead desprezível
Total ponta a ponta (CPU)	20–50 s	100%	Uso interno (<50 requisições/dia)
Total ponta a ponta (GPU)	2–12 s	100%	Produção multiusuário (faixa depende da GPU)

Recomendação por Cenário de Uso

Tabela 9: Recomendação de hardware por cenário de uso

Cenário	Requisições/dia	Hardware	Justificativa
Piloto / testes internos	<20 req/dia	Piso técnico: 7 cores, 22 GB RAM, sem GPU	Prova de conceito ponta a ponta; latência elevada e baixa concorrência
Uso interno (equipe técnica)	20–100 req/dia	8 vCPU, 32 GB RAM, sem GPU	Operação econômica para homologação e uso interno controlado; filas em picos
Produção (SIGESCANTT)	100–500 req/dia	12 vCPU, 48 GB RAM, 1x T4 (16 GB VRAM)	Faixa típica de 6–12 s e melhor atendimento de concorrência
Produção de alta escala	>500 req/dia	16 vCPU, 64 GB RAM, GPU dedicada (24+ GB VRAM) ou múltiplas T4	Faixa típica de 2–5 s em benchmark controlado, com maior throughput

Conclusão sobre hardware: Para a fase inicial de implantação (piloto com a equipe da GEGIR/SUOD), a execução em **CPU é suficiente**. A latência de 20–50 segundos por consulta, embora perceptível, permite validar a qualidade das respostas e ajustar parâmetros sem custo adicional de GPU. Quando o sistema for integrado ao SIGESCANTT para uso em produção com múltiplos usuários, a migração para GPU é **fortemente recomendada**: com T4, a latência típica fica na faixa de 6–12 segundos; com GPU dedicada de maior desempenho, pode chegar a 2–5 segundos em benchmark controlado.

4.4 Pod SIGESCANTT (Sistema Externo)

O SIGESCANTT é o **sistema corporativo existente** da ANTT, gerenciado por outra equipe. Ele possui sua própria stack tecnológica e interface de usuário. A única mudança é a adição de um **módulo de consulta** que faz chamadas HTTP para a API do RAG. A integração com o RAG é feita exclusivamente via **consumo da API REST**.

Responsabilidades do SIGESCANTT

- Fornecer ao usuário um campo de consulta integrado ao fluxo de trabalho existente;
- Enviar a pergunta do usuário para a API REST do RAG;

- Receber e renderizar a resposta (texto, fontes, metadados) na interface do SIGESCANTT;
- Gerenciar autenticação e autorização dos usuários (o RAG não gerencia acesso).

Desacoplamento

Do ponto de vista do SIGESCANTT, o RAG é uma **caixa preta**: ele envia uma pergunta em JSON e recebe uma resposta em JSON. O time do SIGESCANTT **não precisa conhecer** FAISS, embeddings, LangChain, Ollama ou qualquer detalhe de IA. A integração se resume a:

1. Fazer POST para `http://rag-api-svc:8000/api/query` com o corpo JSON;
2. Interpretar o JSON de resposta;
3. Exibir na tela.

O endereço `rag-api-svc:8000` é o **Service interno do Kubernetes** — funciona como um DNS interno que resolve automaticamente para o pod correto. A comunicação é interna ao cluster, sem exposição à internet.

4.5 FAISS Vectorstore — Persistent Volume

O vectorstore não é um pod — é um **volume de disco persistente** (PVC no Kubernetes). Ele armazena os dois arquivos que já existem no projeto:

- `index.faiss` — o índice vetorial com os embeddings dos documentos;
- `index.pkl` — os metadados associados (tipo, número, ano, caminho, etc.).

Esse volume é **montado** nos pods RAG-API e Streamlit no caminho `/app/vectorstore_local/`. Quando a função `carregar_vectorstore_com_provider()` executa, ela lê os arquivos desse volume. Como o FAISS é **somente-leitura** durante consultas, múltiplos pods podem ler simultaneamente sem conflito.

A persistência garante que, se o pod RAG-API reiniciar, o índice não precisa ser recriado — ele sobrevive no disco.

5 Contrato da API REST

A API REST do RAG expõe os seguintes endpoints. O SIGESCANTT (e qualquer outro sistema) consome exclusivamente estes contratos.

5.1 Endpoints

Tabela 10: Endpoints da API REST do RAG

Endpoint	Método	Descrição
/api/health	GET	Health check para probes do Kubernetes (liveness/readiness). Retorna 200 OK se o sistema está operacional.
/api/status	GET	Status detalhado: vectorstore carregado, LLM acessível, quantidade de documentos, provedor de embeddings.
/api/query	POST	Endpoint principal. Recebe pergunta, executa pipeline RAG completo, retorna resposta com fontes.
/api/reindex	POST	Dispara reindexação dos documentos no vectorstore. Operação assíncrona.
/api/documents	GET	Lista documentos indexados com metadados (tipo, número, ano, caminho).
/api/docs	GET	Documentação interativa Swagger/OpenAPI (gerada automaticamente pelo FastAPI).

5.2 Contrato do Endpoint Principal (POST /api/query)

Requisição (JSON)

Listing 3: Corpo da requisição para o endpoint /api/query

```
{
  "pergunta": "Quais os requisitos para autorizacao de transporte
               interestadual de passageiros?",
  "filtros": {
    "tipo_documento": "Resolucao",
    "ano": 2024
  },
  "max_documentos": 5,
  "temperatura": 0.1
}
```

Tabela 11: Campos da requisição

Campo	Tipo	Obrigatório	Descrição
pergunta	string	Sim	Pergunta em linguagem natural
filtros	object	Não	Filtros por metadados (tipo, ano, etc.)
max_documentos	integer	Não	Número máximo de documentos a considerar (padrão: 5)
temperatura	float	Não	Temperatura do LLM, de 0.0 a 1.0 (padrão: 0.1)

Resposta (JSON)

Listing 4: Corpo da resposta do endpoint /api/query

```
{
  "resposta": "Segundo a Resolucao ANTT 5.232/2016, os requisitos
               para autorizacao incluem...",
  "modelo_usado": "qwen2.5:7b",
  "provider": "ollama",
  "documentos_consultados": [
    {
      "tipo": "Resolucao",
      "numero": "5232",
      "ano": "2016",
      "trecho": "Art. 3o Para obtencao de autorizacao...",
      "caminho": "dados_antt/RES/2016/RES-00005232-2016.html",
      "relevancia": 0.87
    }
  ],
  "tempo_processamento_ms": 1250,
  "total_documentos_encontrados": 5,
  "embedding_provider": "local",
  "vectorstore_utilizado": "vectorstore_local"
}
```

Tabela 12: Campos da resposta

Campo	Tipo	Descrição
resposta	string	Texto gerado pelo LLM com citações e referências
modelo_usado	string	Nome do modelo LLM utilizado
provider	string	Provedor do LLM (“ollama”, “deepseek”, “openai”)
documentos_consultados	array	Lista de documentos utilizados como contexto
tempo_processamento_ms	integer	Tempo total de processamento em milissegundos
total_documentos_encontrados	integer	Quantidade de documentos encontrados na busca
embedding_provider	string	Provedor de embeddings utilizado
vectorstore_utilizado	string	Caminho do vectorstore consultado

6 Fluxo de Comunicação

6.1 Fluxo Principal — Consulta via SIGESCANTT

O fluxo completo de uma consulta iniciada pelo usuário no SIGESCANTT percorre os seguintes passos:

- Passo 1. Usuário digita pergunta no SIGESCANTT:** O módulo de consulta RAG, integrado à interface do SIGESCANTT, captura a pergunta em linguagem natural;
- Passo 2. SIGESCANTT envia requisição HTTP:** O frontend do SIGESCANTT faz POST `http://rag-api-svc:8000/api/query` com a pergunta e filtros opcionais. A comunicação é interna ao cluster (ClusterIP), sem exposição à internet;
- Passo 3. RAG-API recebe a requisição:** O FastAPI valida os parâmetros e inicia o pipeline;
- Passo 4. Geração de embedding da pergunta:** O sentence-transformers (executando localmente dentro do pod RAG-API) transforma a pergunta em um vetor de 384 dimensões. Esse processo utiliza a classe `LocalEmbeddings` já existente no código, com o modelo `all-MiniLM-L6-v2`;
- Passo 5. Busca semântica no FAISS:** O vetor da pergunta é comparado com os vetores dos documentos indexados. A função `pesquisar_documentos()` executa busca

MMR (*Maximal Marginal Relevance*) com $k = 12$ e `fetch_k=36`, garantindo diversidade nos resultados. Adicionalmente, uma busca por keywords é executada para complementar os resultados semânticos;

Passo 6. Reranking: Os trechos recuperados passam pela função `reranking_documentos()`, que reordena os resultados com base em correspondência de keywords, relevância técnica, presença de tabelas e priorização de documentos explicitamente citados na pergunta;

Passo 7. Montagem do contexto: Os trechos reordenados são formatados em um prompt estruturado, incluindo metadados de cada documento (tipo, número, ano, seção). Cada trecho é precedido por [Documento: Tipo Número/Ano - Parte X/Y] para rastreabilidade;

Passo 8. Chamada ao Ollama: O pod RAG-API envia o prompt para `http://ollama-svc:11434/v1/chat/completions` com o modelo configurado (ex.: `qwen2.5:7b`). A chamada utiliza a classe `ChatOpenAI` do LangChain — a mesma classe usada atualmente para o DeepSeek, apenas com URL diferente;

Passo 9. Ollama gera a resposta: O modelo processa o contexto e gera uma resposta em português, com citações aos documentos fornecidos. Se a resposta inicial for insatisfatória, o sistema tenta uma extração agressiva de informações (mecanismo já implementado em `gerar_resposta()`);

Passo 10. RAG-API formata a resposta: A resposta bruta do LLM é enriquecida com metadados, fontes, tempo de processamento e informações de rastreabilidade;

Passo 11. Resposta retorna ao SIGESCANTT: O JSON estruturado é enviado de volta ao SIGESCANTT, que renderiza a resposta na interface do usuário.

Todo esse fluxo acontece **dentro do cluster**, sem nenhum dado saindo para a internet. Custo por consulta: **zero** (LLM local + embeddings locais).

6.2 Fluxo Alternativo — Testes via Streamlit

O Streamlit executa exatamente o mesmo pipeline descrito acima, porém de forma integrada: a interface visual chama diretamente as funções Python do RAG (sem passar pela API REST). Isso é possível porque ambos compartilham o mesmo código-fonte e os mesmos volumes de dados.

Tabela 13: Comparação dos fluxos de consulta

Aspecto	Via SIGESCANTT	Via Streamlit
Entrada	HTTP POST (JSON)	Interface visual (formulário)
Autenticação	Gerenciada pelo SIGESCANTT	Acesso direto (rede interna)
Formato de saída	JSON estruturado	Renderização Markdown na tela
Depuração	Logs da API	Métricas visuais, debug na tela
Público-alvo	Usuários finais	Equipe técnica

7 Infraestrutura Kubernetes (Rancher)

7.1 Namespace e Services

Todos os componentes são implantados no namespace **rag-antt**. A comunicação entre pods ocorre via Kubernetes Services internos (ClusterIP), sem exposição direta à internet. Tudo rodar dentro de um único namespace significa:

- **Isolamento de rede:** Pods de outros namespaces não acessam o Ollama;
- **Isolamento de recursos:** Limites de CPU/RAM são definidos por namespace;
- **Gestão centralizada:** Todos os componentes são visíveis e gerenciáveis no painel do Rancher;
- **Deploy atômico:** É possível atualizar ou reiniciar todo o namespace de uma vez.

Tabela 14: Services Kubernetes da arquitetura

Service	Porta	Tipo	Pod alvo	Acesso
sgescantt-svc	80	Ingress	sgescantt	Externo (usuários)
rag-api-svc	8000	ClusterIP	rag-api	Interno (pods)
streamlit-svc	8501	ClusterIP	streamlit	Interno + Ingress opcional
ollama-svc	11434	ClusterIP	ollama	Interno apenas

Observação sobre segurança: O pod Ollama **nunca é exposto** para fora do cluster. Apenas os pods RAG-API e Streamlit podem acessar o LLM via **ollama-svc:11434**. Isso impede uso indevido do modelo por agentes externos.

7.2 Ingress Controller

Nesta subseção, são apresentadas apenas as regras de roteamento por *path*. O papel arquitetural e os controles de segurança do Ingress já foram descritos na seção “Camada de Entrada — Ingress Controller”.

Tabela 15: Regras de roteamento do Ingress

Path externo	Service destino	Observação
/sgescantt	sgescantt-svc:XX	Interface principal
/rag-teste	streamlit-svc:8501	Acesso restrito (equipe técnica)
/api/rag/*	rag-api-svc:8000	Opcional (se sistemas externos ao cluster consumirem)

7.3 Volumes Persistentes

Os dados do sistema (índice vetorial, modelos do Ollama, documentos fonte) são armazenados em Persistent Volume Claims (PVCs) para garantir persistência entre reinicializações de pods:

Tabela 16: Volumes persistentes da arquitetura

PVC	Montagem	Pods	Conteúdo
vectorstore-data	/app/vectorstore_local/	rag-api, streamlit	Índice FAISS (index.faiss + index.pkl)
ollama-models	/root/.ollama/	ollama	Cache dos modelos LLM baixados
dados-antt	/app/dados_antt/	rag-api, streamlit	Documentos fonte (HTML, PDF, JSON)

Nota sobre modo de acesso: O PVC `vectorstore-data` deve ser `ReadWriteMany` (RWX) se ambos os pods (`rag-api` e `streamlit`) precisarem montar o mesmo volume simultaneamente. Caso o provedor de storage não suporte RWX, uma alternativa é replicar o índice em cada pod ou utilizar um volume `ReadOnlyMany` (ROX) para leitura compartilhada.

8 Requisitos de Infraestrutura

8.1 Requisitos do Cluster Kubernetes

Tabela 17: Recursos mínimos estimados para o cluster (baseline técnico)

Pod	CPU	RAM	Observações
rag-api	2 cores	4 GB	sentence-transformers + FAISS em memória
streamlit	1 core	2 GB	Mesmo que o RAG-API, menor carga
ollama (CPU)	4 cores	16 GB	Modelo 7B quantizado (Q4) em CPU
ollama (GPU)	2 cores + GPU	8 GB + 8 GB VRAM	Modelo 7B com aceleração GPU
Total mínimo (CPU)	7 cores	22 GB	Sem GPU
Total mínimo (GPU)	5 cores + GPU	14 GB + 8 GB VRAM	Com GPU

Tabela 18: Dimensionamento recomendado com folga operacional

Cenário	CPU	RAM	GPU	Uso indicado
Inicial (econômico)	8 vCPU	32 GB	Não	Piloto e homologação com baixo volume; maior latência e menor concorrência.
Intermediário (suficiente)	12 vCPU	48 GB	1x GPU T4 (16 GB VRAM)	Produção inicial com melhor experiência e redução significativa de latência.
Ideal (produção)	16 vCPU	64 GB	1x GPU dedicada (16–24+ GB VRAM)	Produção com maior folga operacional, melhor concorrência e menor latência.

Observação: A Tabela 17 apresenta o piso técnico de funcionamento. A Tabela 18 apresenta o dimensionamento sugerido para operação estável, com margem para cres-

cimento e picos.

8.2 Viabilidade do Piloto sem GPU e Riscos Operacionais

Diretriz para início do piloto: É tecnicamente viável iniciar sem GPU, desde que o objetivo seja validação funcional e de qualidade de resposta em baixa carga. O piso técnico indicado neste documento (**7 cores e 22 GB RAM**, sem GPU) permite executar o fluxo completo de ponta a ponta (consulta, recuperação semântica, inferência e retorno de resposta).

Para um piloto de baixa demanda (**até aproximadamente 20 requisições/dia**), a operação tende a ser suficiente para prova de conceito. Entretanto, a experiência de uso é inferior ao cenário com GPU, principalmente por conta da latência de inferência em CPU.

Tabela 19: Limites e riscos do piloto sem GPU (CPU-only)

Aspecto	Limitacao/Risco	Impacto pratico no piloto
Latência por consulta	Faixa típica de dezenas de segundos em CPU (podendo variar para mais em horário de pico)	Menor fluidez para usuário; maior tempo de espera por resposta
Concorrência	Baixa capacidade de atender consultas simultâneas sem fila	Aumento de tempo de resposta quando houver mais de um usuário consultando ao mesmo tempo
Picos de uso	Sensibilidade a variações de carga e tamanho de prompt	Oscilação de desempenho, com consultas mais longas em cenários de contexto extenso
Escalabilidade imediata	Crescimento de demanda pressiona CPU e memória rapidamente	Necessidade de ajuste de capacidade em prazo curto, caso o piloto tenha adesão maior que a prevista

Recomendação operacional: O mínimo (7 cores/22 GB) funciona para provar conceito. Para piloto com uso contínuo por equipe, recomenda-se iniciar ao menos no patamar **8 vCPU e 32 GB RAM** (Tabela 18, cenário “Inicial (econômico)”). Para menor risco de fila e maior estabilidade, o patamar **16 vCPU e 64 GB RAM** (cenário “Ideal (produção)”) oferece folga operacional adicional.

É possível seguir sem GPU na fase inicial, com plena transparência de limites operacionais. Caso a adoção aumente (mais usuários simultâneos, mais consultas por dia ou necessidade de menor tempo de resposta), a migração para GPU deve ser tratada como próxima etapa natural do projeto.

8.3 Requisitos de Disco

Tabela 20: Espaço em disco mínimo estimado (baseline técnico)

Componente	Espaço	Conteúdo
Modelo Ollama (Qwen2.5:7b)	4,7 GB	Pesos do modelo quantizado
Vectorstore FAISS	50–200 MB	Índice vetorial dos documentos
Documentos fonte	500 MB–2 GB	PDFs, HTMLs, JSONs da ANTT
Imagem Docker RAG-API	2–3 GB	Python + dependências
Imagem Docker Ollama	1 GB	Runtime do Ollama
Total estimado	10–15 GB	—

Tabela 21: Capacidade de armazenamento recomendada com folga

Volume/PVC	Capacidade recomendada	Finalidade
ollama-models	30 GB	Modelos LLM locais e cache de versões
vectorstore-data	20 GB	Índice FAISS, metadados e snapshots de reindexação
dados-antt	80 GB	Base documental (PDF/HTML/-Markdown) e crescimento
logs-metricas	20 GB	Logs operacionais e métricas de observabilidade
backup	100 GB	Cópias de segurança de documentos e índices
Total provisionado recomendado	250 GB SSD	Operação com margem para crescimento e retenção

Observação: O total de 250 GB SSD inclui folga operacional para expansão da base, retenção de logs e rotinas de backup, reduzindo risco de indisponibilidade por falta de espaço.

9 Segurança e Isolamento

A arquitetura implementa segurança em camadas, separando rede externa e interna:

9.1 Camada Externa (Usuários)

- O acesso dos usuários é feito exclusivamente via Ingress, com TLS (HTTPS);
- A autenticação e autorização são gerenciadas pelo SIGESCANTT (SSO, LDAP ou mecanismo corporativo existente);
- O Streamlit pode ser protegido por autenticação básica ou restrito a IPs internos via Ingress annotations.

9.2 Camada Interna (Cluster)

- A API REST (`rag-api-svc`) é acessível apenas dentro do cluster (ClusterIP);
- O Ollama (`ollama-svc`) é acessível apenas dentro do cluster — **nunca é exposto externamente**;
- Network Policies do Kubernetes podem restringir comunicação: apenas pods do namespace `rag-antt` podem acessar o `ollama-svc`;
- Nenhum dado regulatório trafega para servidores externos — LLM e embeddings são 100% locais.

9.3 Compliance e Dados

Tabela 22: Análise de compliance por componente

Componente	Dados saem do cluster?	Observação
Embeddings (sentence-transformers)	Não	Modelo local, execução no pod
LLM (Ollama)	Não	Modelo local, execução no pod
Vectorstore (FAISS)	Não	Armazenado em PVC local
Documentos fonte	Não	Armazenados em PVC local
Logs e métricas	Configurável	Podem ser enviados para Prometheus/-Grafana interno

Conclusão: Com LLM e embeddings locais, **nenhum dado regulatório sai do perímetro institucional**. Isso atende requisitos de compliance mesmo em cenários restritivos.

10 Estratégia de Deploy

10.1 Ordem de Implantação

A implantação deve seguir a ordem de dependências:

- Fase 1. PVCs e Ollama:** Criar os volumes persistentes e deployar o pod Ollama com o modelo desejado. Validar que o Ollama responde em `ollama-svc:11434`. Esse é o primeiro passo porque todos os demais dependem do LLM estar disponível;
- Fase 2. RAG-API:** Deployar o pod FastAPI. Validar que o health check responde em `rag-api-svc:8000/api/health` e que a consulta funciona via `/api/query`. Testar pelo menos uma consulta completa (embedding → FAISS → Ollama → resposta);
- Fase 3. Streamlit:** Deployar o pod Streamlit. Validar que a interface carrega em `streamlit-svc:8501` e que as consultas funcionam. Comparar os resultados com os obtidos via API para garantir consistência;
- Fase 4. Integração SIGESCANTT:** Configurar o SIGESCANTT para consumir `rag-api-svc:8000/api/query`. Validar o fluxo completo de ponta a ponta, incluindo a renderização da resposta na interface do SIGESCANTT.

10.2 Fallback e Resiliência

O sistema deve manter operação degradada em caso de falha parcial:

Tabela 23: Cenários de falha e comportamento esperado

Componente em falha	Impacto	Ação automática
Ollama (LLM)	Não gera respostas em linguagem natural	Retorna apenas os trechos relevantes sem sumarização. O RAG opera como busca pura.
Vectorstore (FAISS)	Não realiza busca semântica	Retorna erro 503 Service Unavailable com mensagem explicativa.
RAG-API	Nenhuma consulta funciona	Kubernetes reinicia o pod automaticamente (liveness probe).
Streamlit	Interface de teste indisponível	Não afeta produção (SIGES-CANTT continua via API).

11 Estimativa de Esforço

A transição da aplicação monolítica Streamlit para a arquitetura descrita neste documento envolve as etapas de implementação detalhadas a seguir. O prazo estimado para entrega é de **30 dias úteis**.

11.1 Delimitacao de Escopo para Cronograma

Para fins de planejamento no cronograma do projeto, o escopo do RAG deve ser tratado como um conjunto de entregas técnicas relacionadas a ingestão de base de conhecimento textual, indexação semântica, consulta via API e suporte a integração com o SIGESCANTT. O RAG não deve ser apresentado como componente multimodal, nem como motor operacional diretamente conectado aos parâmetros de negócio do SIGESCANTT.

Tabela 24: Premissas e limites do escopo do RAG para o cronograma

Item	Dentro do escopo	Fora do escopo
Base de conhecimento	Upload, armazenamento, extração de texto, chunking, embeddings e indexação de documentos textuais. Inclui PDF, HTML, TXT, Markdown, JSON textual e planilhas convertidas para representação textual/tabular.	Processamento multimodal de imagem, áudio, vídeo, OCR avançado ou interpretação visual de gráficos e figuras.
Planilhas	Permitido quando a planilha for usada como fonte textual ou tabular de conhecimento, por exemplo CSV ou XLSX convertido para linhas, colunas e metadados pesquisáveis.	Uso da planilha como sistema transacional, calculadora operacional oficial ou fonte dinâmica vinculada automaticamente aos parâmetros ativos do SIGESCANTT.
Parâmetros operacionais	Documentos textuais relacionados a parâmetros operacionais podem ser carregados e consultados, desde que entrem como conhecimento documental versionado.	Vínculo direto via API a parâmetros operacionais vivos, regras de negócio executáveis, simuladores, motores de cálculo ou bases transacionais do SIGESCANTT.
Consulta pelo SIGESCANTT	API REST para receber pergunta, executar o pipeline RAG e devolver resposta, fontes, trechos e metadados para exibição ao usuário do SIGESCANTT.	Alterações na interface, autenticação, autorização, regras internas ou persistência transacional do SIGESCANTT.
Resultado da query	Retorno estruturado em JSON pelo endpoint de consulta, permitindo que o SIGESCANTT apresente a resposta ao usuário e registre metadados conforme sua própria regra.	Decisão automática, homologação jurídica automática, atualização de cadastros ou execução de ações operacionais no SIGESCANTT.

11.2 Estrutura de Etapas e Subetapas

A Tabela 25 organiza o escopo em formato de etapas e subetapas para uso direto em ferramenta de cronograma. As atividades foram descritas como entregas verificáveis, evitando ambiguidade sobre o que o RAG faz e o que permanece como responsabilidade de sistemas externos.

Tabela 25: Etapas e subetapas sugeridas para o cronograma do RAG – parte 1

Etapa	Subetapa	Descricao objetiva	Entregavel
1. Delimitacao funcional do RAG	1.1 Definir tipos de fonte textual	Formalizar que o RAG trabalhara com base de conhecimento textual: PDF, HTML, TXT, Markdown, JSON textual e planilhas convertidas para texto ou tabela pesquisavel.	Matriz de fontes aceitas e nao aceitas.
1. Delimitacao funcional do RAG	1.2 Definir exclusoes de multimodalidade	Registrar que o RAG nao processara imagem, audio, video, leitura visual de figuras, OCR avancado ou interpretacao de graficos como dado visual.	Declaracao de nao multimodalidade no escopo.
1. Delimitacao funcional do RAG	1.3 Definir limite sobre parametros operacionais	Registrar que parametros operacionais podem ser consultados apenas quando estiverem descritos em documentos textuais carregados na base. Nao havera conexao direta com parametros operacionais vivos via API.	Regra de fronteira entre conhecimento documental e sistema operacional.
2. Ingestao da base de conhecimento	2.1 Implementar upload de documentos textuais	Disponibilizar mecanismo para receber documentos textuais e armazenar os arquivos em area controlada para processamento.	Upload funcional para documentos suportados.
2. Ingestao da base de conhecimento	2.2 Implementar upload de planilhas	Permitir upload de planilhas quando seu conteudo puder ser convertido para texto estruturado, preservando nome de abas, colunas, linhas e metadados relevantes.	Conversao de CSV/XLSX para texto indexavel.
2. Ingestao da base de conhecimento	2.3 Validar arquivos recebidos	Validar extensao, tamanho, tipo, conteudo extraivel e erros de leitura antes de inserir o documento na base.	Rotina de validacao e relatorio de falhas.
3. Processamento e indexacao	3.1 Extrair e normalizar texto	Extrair conteudo textual, remover ruido, padronizar metadados e preparar o material para chunking.	Textos normalizados com metadados.
3. Processamento e indexacao	3.2 Gerar chunks e embeddings	Dividir documentos em trechos, gerar embeddings locais e preparar os vetores para busca semantica.	Chunks vetorizados.
3. Processamento e indexacao	3.3 Atualizar indice vetorial	Persistir ou atualizar o indice FAISS com os documentos processados.	Indice vetorial atualizado e consultavel.

Tabela 26: Etapas e subetapas sugeridas para o cronograma do RAG – parte 2

Etapa	Subetapa	Descricao objetiva	Entregavel
4. Consulta RAG via API	4.1 Implementar endpoint de consulta	Receber pergunta, filtros opcionais e parametros de busca pelo endpoint <code>/api/query</code> .	Contrato REST de consulta.
4. Consulta RAG via API	4.2 Executar recuperacao e geracao	Buscar trechos relevantes, montar contexto, chamar o LLM local e gerar resposta em linguagem natural.	Pipeline RAG ponta a ponta.
4. Consulta RAG via API	4.3 Retornar resposta auditavel	Retornar resposta, fontes, trechos, relevancia, tempo de processamento e metadados em JSON.	Resposta estruturada para consumo pelo SIGES-CANTT.
5. Integracao com SIGES-CANTT	5.1 Apoiar consumo da API pelo SIGESCANTT	Fornecer contrato, exemplos de requisicao e orientacao tecnica para que o SIGES-CANTT consulte o RAG.	Guia de integracao e exemplos HTTP/JSON.
5. Integracao com SIGES-CANTT	5.2 Validar consulta pelo usuario final	Validar que o usuario do SIGESCANTT consegue enviar uma pergunta e receber a resposta retornada pela API do RAG.	Fluxo de consulta validado em homologacao.
5. Integracao com SIGES-CANTT	5.3 Registrar limites de responsabilidade	Confirmar que o RAG responde perguntas com base documental, mas nao altera dados, nao executa calculos oficiais e nao atualiza parametros operacionais no SIGESCANTT.	Termo tecnico de limites da integracao.
6. Testes e homologacao	6.1 Testar formatos aceitos	Testar documentos textuais e planilhas convertidas para texto, incluindo casos invalidos.	Evidencias de teste de ingestao.
6. Testes e homologacao	6.2 Testar qualidade das respostas	Avaliar respostas com perguntas reais, conferindo fontes, trechos e aderencia ao conteudo documental.	Relatorio de qualidade do RAG.
6. Testes e homologacao	6.3 Testar API e desempenho	Validar disponibilidade, tempo de resposta, erros esperados e comportamento sob carga controlada.	Relatorio de testes tecnicos.

Tabela 27: Estimativa de esforço para implementação (escopo DeepFeed)

Etapa	Descrição	Horas	Semana
1	Refatorar <code>antt_rag_unified.py</code> : extrair lógica de busca, geração de resposta e reranking para módulos Python independentes, reutilizáveis tanto pela API quanto pelo Streamlit	32	1–2
2	Desenvolver API FastAPI com todos os endpoints (query, health, status, reindex, documents), validação Pydantic, tratamento de erros e documentação Swagger	40	2–3
3	Adicionar provedor Ollama no <code>config.py</code> e <code>llm_providers.py</code> , incluindo fallback para outros provedores em caso de indisponibilidade	16	3
4	Criar Dockerfiles otimizados para RAG-API e Streamlit, incluindo multi-stage build, cache de dependências e testes de build	16	4
5	Criar manifests Kubernetes (Deployments, Services, PVCs, Ingress, ConfigMaps, resource limits), com ambientes de staging e produção	24	4–5
6	Testes de integração ponta a ponta: API + Ollama + vectorstore + Streamlit, incluindo testes de carga, concorrência e cenários de falha	32	5
7	Documentação técnica da API para equipe OTI: contrato, exemplos de chamada, guia de integração e tratamento de erros	16	6
8	Suporte à integração SIGEScantT: acompanhamento técnico, resolução de dúvidas e ajustes durante a integração	24	6
Total (escopo DeepFeed)		200 h	30 dias

11.3 Etapas Dependentes da ANTT

As seguintes atividades **não estão incluídas** na estimativa acima por dependerem de ações, decisões e infraestrutura da equipe da ANTT:

Tabela 28: Atividades dependentes da ANTT (fora do escopo de estimativa)

Atividade	Dependência
Provisionar cluster Kubernetes (Rancher)	Equipe de infraestrutura da ANTT deve disponibilizar o cluster com recursos adequados (CPU, RAM, GPU opcional, storage)
Configurar Ollama no Rancher (pod + PVC + modelo)	Requer acesso ao cluster, definição de <i>resource quotas</i> , download do modelo LLM e eventual configuração de GPU — depende da infraestrutura disponibilizada pela ANTT
Configuração de rede e Ingress	Definição de domínios, certificados TLS, regras de firewall e políticas de rede interna — responsabilidade da equipe de redes/segurança da ANTT
Integração com autenticação corporativa	SSO, LDAP ou mecanismo de autenticação do SIGESCANTT — responsabilidade da equipe do OTI
Homologação e aceite em produção	Validação funcional, aceite de qualidade e aprovação para publicação em ambiente de produção

11.4 Escopo Não Incluído — Integração com Crawler

Importante: A estimativa de 30 dias úteis **não contempla** o desenvolvimento e integração do **Crawler de atualização da base de conhecimento**.

O Crawler é um componente complementar cuja função é realizar varreduras periódicas nos portais da ANTT e do DNIT (e eventualmente outros, como sites de concessionárias), extraindo documentos regulatórios atualizados (Resoluções, Instruções Normativas, Notas Técnicas, Deliberações, etc.) em formato PDF e/ou Markdown. Esses documentos seriam então enviados ao RAG para reindexação automática, mantendo a base de conhecimento sempre atualizada sem intervenção manual.

O fluxo previsto para a integração Crawler–RAG é:

1. O Crawler executa periodicamente (ex.: diariamente ou semanalmente) a varre-

dura dos portais da ANTT e DNIT;

2. Documentos novos ou atualizados são baixados e armazenados em um diretório compartilhado ou enviados via API;
3. O RAG detecta os novos documentos (via webhook, polling ou chamada direta ao endpoint `POST /api/reindex`);
4. O pipeline de ingestão processa os documentos (extração de texto, chunking, geração de embeddings);
5. O índice FAISS é atualizado com os novos vetores, tornando os documentos disponíveis para consultas.

Essa integração envolve desenvolvimento adicional que inclui: adaptação do Crawler existente, criação do pipeline de ingestão automatizado, tratamento de duplicatas e versionamento de documentos, monitoramento de falhas no Crawler, e testes de reindexação incremental.

11.5 Resumo do Escopo

Tabela 29: Resumo do escopo da estimativa

Item	Incluído?
Refatoração do código e criação da API REST	Sim
Provedor Ollama (LLM local)	Sim
Dockerfiles e manifests Kubernetes	Sim
Testes de integração e documentação	Sim
Suporte à integração com SIGESCANTT	Sim
Provisão de infraestrutura no Rancher (cluster, rede, GPU)	Não (ANTT)
Configuração do Ollama no cluster	Não (ANTT)
Desenvolvimento/integração do Crawler de atualização	Não (escopo separado)
Customizações de interface no SIGESCANTT	Não (equipe OTI)

12 Conclusão

A arquitetura proposta transforma o Sistema RAG-ANTT de uma aplicação monolítica Streamlit em um conjunto de microsserviços orquestrados pelo Kubernetes (Rancher), mantendo compatibilidade total com o código existente. Os principais ganhos são:

1. **Desacoplamento completo:** A lógica de RAG é exposta como API REST, consumível por qualquer sistema (SIGESCANTT, aplicativos móveis, chatbots, automações);
2. **Custo zero de inferência:** Com Ollama e sentence-transformers locais, não há cobrança por chamada a APIs externas. O custo se limita à infraestrutura computacional;
3. **Compliance total:** Nenhum dado regulatório sai do perímetro do cluster. LLM, embeddings e vectorstore operam 100% localmente;
4. **Continuidade de testes:** O Streamlit permanece disponível para a equipe técnica, sem impactar a produção;
5. **Resiliência:** Kubernetes garante reinício automático de pods, e a arquitetura suporta operação degradada em caso de falha do LLM;
6. **Evolução incremental:** A transição pode ser feita em fases, começando pelo deploy da API e Ollama, e integrando o SIGESCANTT em um segundo momento.

A recomendação é iniciar com um **teste local** do Ollama para validar a qualidade do modelo escolhido com documentos reais da ANTT, e então replicar a configuração no ambiente Rancher seguindo a ordem de implantação descrita na Seção 9.

Este documento foi elaborado como referência de arquitetura para o deploy do Sistema RAG-ANTT. A arquitetura descrita é a recomendação técnica atual e pode ser revisada conforme evolução dos requisitos, infraestrutura disponível e resultados do piloto.